

# Representación compacta a nivel de bits y gestión dinámica de memoria para el juego *Sweet Crush*

## Informe de análisis y diseño de la solución

---

*Documento de análisis y diseño. Describe la arquitectura lógica y modular de la solución antes de su implementación; no constituye la bitácora de desarrollo.*

---

## 1. Contextualización

*Sweet Crush* pertenece a la familia de juegos *match -3* : un tablero rectangular poblado de fichas de colores en el que el jugador elimina piezas, provoca que las restantes caigan por gravedad y desencadena reacciones en cadena cuando tres o más fichas iguales quedan alineadas. Descrito así, el problema parece un ejercicio de lógica de juego. Esa lectura es engañosa y conviene desmontarla desde el principio, porque orienta mal el esfuerzo.

El verdadero objeto del desafío no es la mecánica visible sino la capa que la sostiene: cómo se guarda el tablero en memoria. La consigna prohíbe lo natural (usar un char o un entero por casilla) y exige que cada posición se represente con exactamente tres bits empaquetados en una secuencia continua sobre un bloque de bytes en memoria dinámica. Seis de las ocho combinaciones de tres bits nombran las seis fichas; las dos restantes quedan libres para estados auxiliares, como una casilla vacía. La consecuencia es que un byte deja de coincidir con una ficha: los tres bits de una pieza pueden repartirse entre dos bytes vecinos, y las fronteras lógicas del tablero dejan de alinearse con las fronteras físicas del almacenamiento.

Esa decisión de representación reordena por completo dónde está la dificultad. La rúbrica reparte la calificación en manipulación de bits (30 %), gestión de memoria (35 %) y lógica de juego (35 %); dos tercios del peso viven, por tanto, en la infraestructura que el jugador nunca ve. La detección de combinaciones y las cascadas son algoritmos conocidos y de riesgo acotado. Lo que hunde una solución es más sutil: una primitiva de acceso a tres bits que falla cuando la ficha cruza un límite de byte, o un redimensionamiento del tablero que corrompe el empaquetado. Este informe asume esa jerarquía y ordena el análisis y el diseño de abajo hacia arriba, desde el bit hacia el juego, porque es el orden en el que se acumula (y se controla) el riesgo real.

La justificación pedagógica del ejercicio se vuelve entonces transparente. Obligar a un empaquetado de tres bits fuerza el dominio efectivo de máscaras y desplazamientos; prohibir el sobredimensionamiento y exigir memoria dinámica ajustada obliga a razonar sobre la aritmética exacta de bytes; vetar struct, class, plantillas y la biblioteca estándar impide delegar el problema y expone las decisiones de bajo nivel que esas herramientas suelen ocultar. El juego es el pretexto; la administración explícita de la memoria y de los bits es la lección.

## 2. Análisis

Depurando la consigna, el sistema debe sostener el siguiente conjunto de operaciones, agrupadas por naturaleza:

- Crear un tablero  $F \times C$  con dimensiones indicadas al inicio; poblarlo con fichas aleatorias de distribución uniforme; mostrarlo en pantalla en dos registros, el de fichas legibles y el binario crudo. Esto es el ciclo de vida del tablero.
- Permitir que el jugador seleccione una posición y elimine la ficha allí ubicada. Esto es la interacción.
- Detectar combinaciones de tres o más fichas iguales, consecutivas en horizontal o en vertical; resolver correctamente los casos en que una ficha pertenece a la vez a una combinación horizontal y a una vertical. Esto hace parte de la resolución.
- Tras cada eliminación, desplazar verticalmente las fichas supervivientes para rellenar huecos e incorporar fichas nuevas hasta recuperar las dimensiones. La aparición de estas piezas puede generar combinaciones sin intervención del jugador. Esto hace parte de la reorganización.
- Reprocesar el tablero mientras sigan surgiendo combinaciones automáticas, hasta alcanzar un estado estable, y registrar cuántas cascadas ocurren.
- Agregar o eliminar filas y columnas en cualquier posición válida, incluidas las intermedias, no solo en los bordes.
- Mantener, como mínimo, dimensiones actuales, eliminaciones del usuario, fichas eliminadas en total, combinaciones detectadas, cascadas de la jugada en curso y puntuación.

## 2.1 Restricciones y sus consecuencias

Cada restricción cierra un camino de implementación y varias tienen efectos que no son evidentes a primera vista. Conviene leerlas como muros, y diseñar contra ellos.

## 2.2. Aritmética del empaquetado

El análisis fija primero las relaciones numéricas que gobiernan la ubicación de cada ficha; el diseño se construye sobre ellas. Para un tablero de  $F$  filas y  $C$  columnas:

$$\text{Posiciones: } N = F \cdot C, \quad (1)$$

$$\text{Bits de datos: } 3N, \quad (2)$$

$$\text{Bytes reservados: } B = \left\lceil \frac{3N}{8} \right\rceil = \left\lceil \frac{3N+7}{8} \right\rceil \quad (3)$$

$$\text{Índice lógico: } i = \text{fila} \cdot C + \text{columna}, \quad (4)$$

$$\text{Bit inicial: } b = 3i, \quad (5)$$

$$\text{Byte y desplazamiento: } K = \frac{b}{8} \quad (6)$$

El bit inicial  $b$  se cuenta desde el bit menos significativo de la tira completa, de modo que los  $3N$  bits válidos ocupan la parte baja del bloque y los  $P = 8B - 3N$  bits de relleno quedan arriba, en la parte más significativa del último byte. Cuando  $d \in \{6, 7\}$ , los tres bits de la ficha no caben en un solo byte y se reparten entre el byte  $k$  y el byte  $k + 1$ : es el caso crítico que la primitiva de acceso debe resolver sin excepción.

El documento contiene una tensión que el análisis debe zanjar de forma explícita. La Figura 2 del enunciado ilustra un empaquetado en el que la primera ficha ocupa los bits altos del byte 0; el apartado C, en cambio, ordena que los bits válidos inicien en el bit menos significativo y que los vacíos se agrupen a la izquierda. Son órdenes físicos opuestos. Este diseño acata el apartado C (es una regla normativa, no una ilustración) y observa que la Figura 2 no lo contradice de hecho: emplea ocho fichas, es decir 24 bits exactos, tres bytes sin relleno, un caso en el que la dirección del *padding* nunca se hace visible. La convención adoptada, con el dato alineado al LSB, es la que se representa en la Figura 1.

| Restricción                                                                                                 | Consecuencia de diseño                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tres bits por casilla; secuencia continua, sin relleno para alinear cada ficha a un byte                    | El acceso a una ficha nunca es una lectura de arreglo: es una extracción de bits que puede abarcar dos bytes. Toda la solución se apoya en dos primitivas atómicas (leer y escribir tres bits) que deben resolver el cruce de frontera.                                                                                          |
| Almacenamiento en unsigned char * dinámico; el byte no representa una ficha                                 | El bloque físico es una tira de bits; la correspondencia (fila, columna) → bits debe calcularse, no indexarse.                                                                                                                                                                                                                   |
| Datos alineados al bit menos significativo; bits vacíos agrupados a la izquierda (apartado C)               | Fija la convención física de empaquetado y, sobre todo, obliga a <i>reubicar el relleno</i> cada vez que cambia $F \times C$ . Es el requisito que convierte el redimensionamiento en un reempaquetado, no en un simple realloc.                                                                                                 |
| Memoria ajustada a las dimensiones actuales; prohibido sobredimensionar o reservar para expansiones futuras | Crecer implica reservar el tamaño exacto y trasladar los datos. Encoger no es inmediato: solo se libera memoria física cuando la utilización cae por debajo del 65 % del tablero, para evitar reasignaciones en cada borrado.                                                                                                    |
| Sin struct, class, template, objetos propios ni STL                                                         | No existe un “objeto tablero”. El estado (puntero, $F$ , $C$ , contadores, puntuación) no puede encapsularse en un agregado: debe vivir como variables de ámbito de módulo, expuestas mediante funciones, o pasarse por referencia entre funciones. Esta es la consecuencia menos obvia y la que más condiciona la arquitectura. |
| Sin sintaxis ANSI C (printf, malloc. . . ); C++ con las limitaciones anteriores                             | Reserva con new[]/delete[], salida con flujos de C++, referencias y bool. Programación estructurada, no orientada a objetos.                                                                                                                                                                                                     |
| Sin objetos string                                                                                          | Los rótulos y la interfaz se manejan con char y arreglos de char; la visualización elige caracteres que hagan legible el movimiento de fichas.                                                                                                                                                                                   |
| Uso esencial de operadores &,  , ^, ~, «, »; prohibido desempaquetar de forma permanente                    | El tablero se interpreta siempre sobre su forma empaquetada. No se admite una copia paralela de un byte por casilla sobre la que operar cómodamente.                                                                                                                                                                             |
| Diseño modular multi-archivo (.h/.cpp)                                                                      | La solución se organiza en unidades de compilación con interfaces mínimas y dependencias en una sola dirección.                                                                                                                                                                                                                  |

La solución es viable con las herramientas exigidas; ningún requisito obliga a algo que los operadores de bits y la memoria dinámica no cubran. El análisis, sin embargo, identifica cuatro puntos donde se concentra la probabilidad de falla, y que el diseño debe blindar antes que cualquier otra cosa:

1. Cruce de frontera de byte en la lectura y escritura de tres bits. Es el cimientito: si estas dos funciones fallan, falla todo lo demás de forma difícil de diagnosticar.
2. Reempaquetado en el redimensionamiento al insertar o eliminar una fila o columna intermedia cambia  $N$ , corre la posición física de casi todas las fichas posteriores y desplaza el relleno. Aquí está la ingeniería de verdad.
3. Combinaciones simultáneas en una ficha compartida por una línea horizontal y una vertical debe eliminarse una sola vez y sin romper la detección de la otra línea.
4. Terminación de las cascadas en el bucle de reprocesamiento debe converger; el diseño debe garantizar que cada iteración progresa hacia un estado estable.

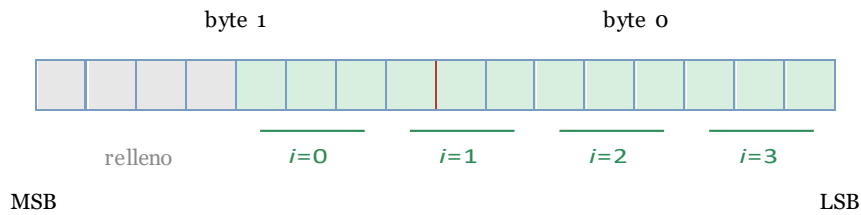


Figura 1: Alineación adoptada para  $N = 4$  ( $3N = 12$  bits,  $B = 2$  bytes,  $P = 4$  bits de relleno). Los datos ocupan la parte baja; el relleno se consolida a la izquierda.

### 3. Diseño

La arquitectura responde directamente a la jerarquía de riesgo. Se organiza en cinco módulos apilados, cada uno un par `.h/.cpp`, con dependencias estrictamente descendentes: un módulo solo conoce al que tiene inmediatamente debajo. Esa unidireccionalidad permite verificar cada capa de forma aislada (empezando por la más baja) antes de construir sobre ella.

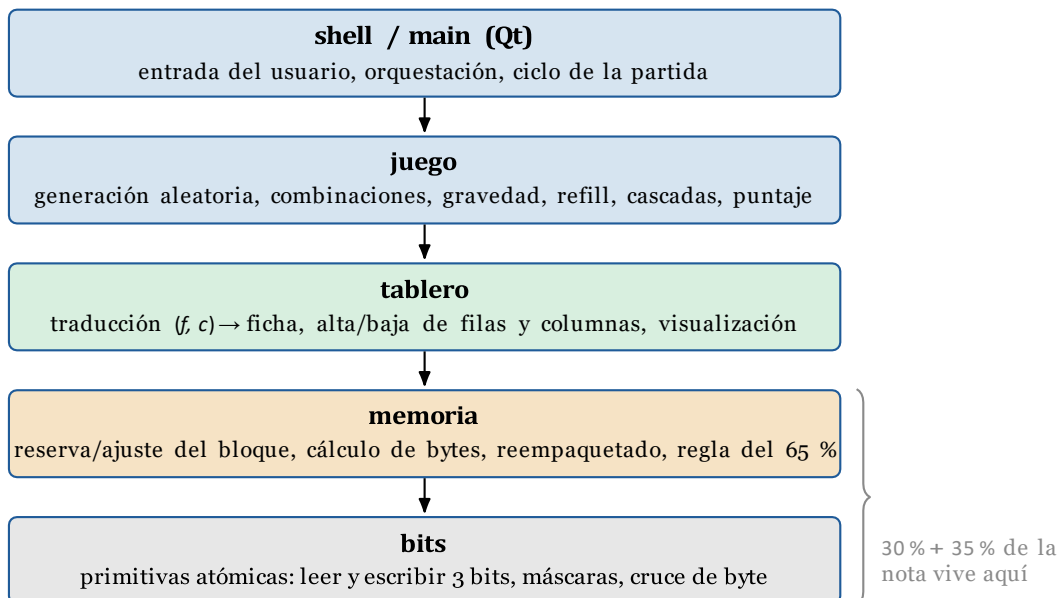


Figura 2: Arquitectura en capas. Las flechas indican la dirección de dependencia. El riesgo y la mayor parte de la calificación se concentran en las dos capas inferiores.

**bits** Núcleo atómico. Ofrece *leer tres bits* y *escribir tres bits* dada una posición inicial dentro del bloque, resolviendo internamente el caso en que la ficha se reparte entre dos bytes. Es la única capa que toca directamente los operadores `&`, `|`, `<<`, `>>` y las máscaras; todo lo demás delega aquí. Se prueba de forma independiente antes de continuar.

**memoria**

Administra el `unsigned char *`: calcula  $B$  a partir de  $F$  y  $C$ , reserva y libera con `new[]/delete[]`, reempaqueta cuando cambian las dimensiones y aplica la histéresis del 65 % al encoger. Custodia el invariante de alineación (datos al LSB, relleno a la izquierda).

**tablero**

Traduce el mundo lógico al físico: de  $(f, c)$  obtiene el índice, de este el bit inicial, y llama a `bits` para leer o escribir la ficha. Implementa el alta y la baja de filas y columnas en

cualquier posición, y la doble visualización (fichas y binario).

juego Reglas de *Sweet Crush*: poblado aleatorio uniforme, detección de combinaciones, gravedad, incorporación de fichas, bucle de cascadas, puntuación y contadores.

shell / main  
Capa de presentación y control en Qt: lee la entrada, invoca las operaciones del juego y muestra el resultado.